

Chapitre 5

Synthèse du Chapitre 5 : Les collections en PL/SQL

Ce chapitre aborde les **collections** en PL/SQL, qui sont des structures de données permettant de stocker plusieurs éléments de même type. Les collections sont utiles pour manipuler des ensembles de données de manière efficace, notamment lors de l'interaction avec des bases de données. PL/SQL propose trois types de collections : les **tableaux associatifs**, les **tables imbriquées** et les **tableaux prédimensionnés**.

1. Définir des types collections

Les collections sont des ensembles ordonnés d'éléments de même type, accessibles via un indice. Elles peuvent être unidimensionnelles, mais il est possible de créer des collections multidimensionnelles en utilisant des collections de collections.

- **Tableaux associatifs (associative arrays ou index-by tables) :**

- Ce sont des collections dont les éléments sont accessibles via un indice de type numérique ou chaîne de caractères.
- Syntaxe :

```
TYPE nom_type IS TABLE OF type_element INDEX BY type_indice;
```

- Exemple :

```
TYPE TypeTableMessErreur IS TABLE OF VARCHAR2(200) INDEX BY  
BINARY_INTEGER;  
TableMessErreur TypeTableMessErreur;
```

- **Tables imbriquées (nested tables) :**

- Ce sont des collections non ordonnées d'éléments indicés par des valeurs numériques consécutives.
- Elles peuvent être stockées dans des tables de base de données.
- Syntaxe :

```
TYPE nom_type IS TABLE OF type_element;
```

- **Tableaux prédimensionnés (variable-size arrays) :**

- Ce sont des collections ordonnées dont la taille est fixée lors de la déclaration, mais peut être modifiée lors de l'exécution.
- Syntaxe :

```
TYPE nom_type IS VARRAY(taille) OF type_element;
```

2. Méthodes associées aux collections

PL/SQL propose plusieurs méthodes pour manipuler les collections :

- **EXISTS** : Vérifie si un élément existe à un indice donné.
 - Exemple :

```
IF LesEmployes.EXISTS(25) THEN  
    DBMS_OUTPUT.PUT_LINE('Élément existe');  
END IF;
```

- **COUNT** : Retourne le nombre d'éléments dans la collection.
 - Exemple :

```
DBMS_OUTPUT.PUT_LINE(LesEmployes.COUNT);
```

- **FIRST et LAST** : Retournent respectivement le premier et le dernier indice de la collection.
 - Exemple :

```
FOR i IN LesEmployes.FIRST..LesEmployes.LAST LOOP  
    DBMS_OUTPUT.PUT_LINE(LesEmployes(i).Nom);  
END LOOP;
```

- **PRIOR et NEXT** : Retournent l'indice précédent ou suivant un indice donné.
 - Exemple :

```
i := LesEmployes.NEXT(i);
```

- **EXTEND** : Augmente la taille d'une collection (non applicable aux tableaux associatifs).

- Exemple :

```
LesEmployes.EXTEND;
```

- **TRIM** : Réduit la taille d'une collection en supprimant des éléments à la fin (non applicable aux tableaux associatifs).

- Exemple :

```
LesEmployes.TRIM;
```

- **DELETE** : Supprime un ou plusieurs éléments d'une collection.

- Exemple :

```
LesEmployes.DELETE(5); -- Supprime l'élément à l'indice 5
```

3. Exceptions liées aux collections

Plusieurs exceptions peuvent être levées lors de l'utilisation des collections :

- **COLLECTION_IS_NULL** : Levée lorsqu'une collection est nulle (non initialisée).
- **NO_DATA_FOUND** : Levée lorsqu'un indice désigne un élément qui n'existe pas ou a été supprimé.
- **SUBSCRIPT_BEYOND_COUNT** : Levée lorsqu'un indice dépasse le nombre d'éléments dans la collection.
- **SUBSCRIPT_OUTSIDE_LIMIT** : Levée lorsqu'un indice est en dehors des limites autorisées.
- **VALUE_ERROR** : Levée lorsqu'un indice est nul ou ne peut pas être converti dans le type de l'indice.

4. Exemples pratiques

- **Initialisation d'un tableau associatif :**

```
DECLARE
    TYPE TypeTableMessErreur IS TABLE OF VARCHAR2(200) INDEX BY
    BINARY_INTEGER;
    TableMessErreur TypeTableMessErreur;
BEGIN
    TableMessErreur(1) := 'Erreur 1';
    TableMessErreur(2) := 'Erreur 2';
END;
```

- **Utilisation de la méthode COUNT :**

```
DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
BEGIN
    SELECT * BULK COLLECT INTO LesEmployes FROM Employees;
    DBMS_OUTPUT.PUT_LINE('Nombre d\\'employés : ' || LesEmployes.
    COUNT);
END;
```

- **Gestion des exceptions :**

```
DECLARE
    TYPE TableEmployes IS TABLE OF Employees%ROWTYPE INDEX BY
    BINARY_INTEGER;
    LesEmployes TableEmployes;
BEGIN
    DBMS_OUTPUT.PUT_LINE(LesEmployes(1).Nom); -- Lève NO_DATA_
    FOUND
    EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aucun employé trouvé');
END;
```

Conclusion

Les collections en PL/SQL sont des outils puissants pour manipuler des ensembles de données. Elles permettent de stocker et de manipuler des éléments de manière efficace, notamment lors de l'interaction avec des bases de données. Les méthodes associées aux collections facilitent leur manipulation, tandis que les exceptions permettent de gérer les erreurs potentielles de manière contrôlée. Une bonne compréhension des collections et de leurs méthodes est essentielle pour écrire des programmes PL/SQL robustes et performants.